

Card Based Dungeon-Crawling Game

Gustavo Antunes, nº50485

André Brito, nº50487

Orientadores: Pedro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Card Based Dungeon-Crawling Game

50485 Gustavo Antunes

50485 André Brito

Orientadores: Pedro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

Agradecimentos

Em primeiro lugar, obrigado a todos os que acreditaram que seria possível executar este projeto, que acreditaram na ideia, e que nos ajudaram ao longo deste percurso, em particular, o orientador Pedro Pereira. Não tendo sido uma jornada sem os seus altos e baixos, o que mais se destacou não foram as dificuldades e problemas, mas o apoio que recebemos ao longo da execução e conclusão deste projeto.

Às outras pessoas que também participaram indiretamente desta jornada, e que a tornaram mais fácil por quaisquer razões, obrigado. Se conseguimos acabar o que conseguimos (a tempo da entrega...), é graças a vocês.

Abstract This report presents the design and implementation of an application that enables the execution of a game conceived and developed as part of the Project and Seminar course. The challenge was to develop an application that allows different types of client to authenticate, create or search for game tables, and enjoy a real-time multiplayer game with a unique gameplay experience that combines several concepts not commonly found together.

A aplicação aborda tópicos essenciais do desenvolvimento de software, como a imutabilidade dos dados, código concorrente, a partilha de código comum a vários componentes de uma aplicação, e a fácil expansibilidade do código escrito totalmente em Kotlin.

Resumo Este relatório apresenta o desenho e implementação de uma aplicação que permite a execução de um jogo planeado e desenvolvido no âmbito da unidade curricular Projeto e Seminário. O desafio era realizar uma aplicação que permitisse que tipos de cliente diferentes fossem capazes de se autenticar, criar ou procurar mesas de jogo, e desfrutar de um jogo multijogador em tempo real com uma vertente única, que mistura vários conceitos diferentes e que não costumam coexistir.

A aplicação aborda tópicos essenciais do desenvolvimento de software, como a imutabilidade dos dados, código concorrente, a partilha de código comum a vários componentes de uma aplicação, e a fácil expansibilidade do código escrito totalmente em Kotlin.

Utilização de Inteligência Artificial

A performance das ferramentas AI tem sido, na sua grande maioria, decepcionante, mesmo restringindo o uso destas ferramentas a tarefas simples, por exemplo, a realização de testes automáticos para elementos do domínio da aplicação, ou para a criação de diagramas PlantUML ou outros elementos de documentação. É importante notar, que quaisquer crítica ou dificuldade é feita à opção gratuita de cada modelo.

É importante ressaltar em primeiro lugar, que há uma diferença significativa no uso das ferramentas AI no browser das que estão instaladas localmente na máquina ou na IDE. O tempo e recursos que são gastos a contextualizar as ferramentas no browser não compensa de todo a sua utilização, ao invés de se gastar tempo a escrever código, usa-se a mesma quantidade de tempo para pedir que a AI gere o código, retificá-lo, e muitas vezes completar.

Fora do contexto de utilização previamente descrito, nenhum modelo, com ou sem contexto, foi capaz de cumprir com todas as exigências que nos tínhamos proposto a seguir, por exemplo, adicionando mutabilidade onde não era de todo necessário.

A utilização correta destas ferramentas é crucial, principalmente nos dias de hoje em que tudo se está a ajustar lentamente a AI, mas também é importante notar que confiar demasiado nestas ferramentas pode contribuir para uma deteriorização do sentido critico, e para uma maior dificuldade em pensar em várias possíveis soluções para um problema de forma independente, o que consideramos ser qualidades essenciais para um engenheiro.

Índice

1	Introdução	1
1.1	O Jogo	1
1.2	Como Jogar	2
1.3	Organização do Documento	3
2	Enquadramento	5
2.1	Trabalho relacionado	5
2.2	Sistemas semelhantes	5
3	Arquitetura	7
3.1	Tecnologias usadas	7
3.2	Aplicações cliente	9
3.3	Aplicação servidor	9
3.4	Base de dados	10
3.5	Requisitos Funcionais	10
4	Implementação	11
4.1	Organização do projeto	11
4.1.1	Compose App	11
4.1.2	Server	11
4.1.3	Shared	12
4.2	FrontEnd/Cliente	12
4.2.1	Navegação	12
4.2.2	ClientAPI	13
4.2.3	ViewModel	14
4.2.4	Ecrãs	14
4.3	BackEnd/Servidor	14
4.3.1	Base de dados	14
4.3.2	Dominio	17
4.3.3	Expansibilidade	17
4.3.4	Autenticação	18

4.3.5	Comunicação cliente-servidor	20
4.3.6	Testes	21
5	Conclusões	23
5.1	Dificuldades no projeto	23
5.2	Objetivos não implementados	23
5.3	Melhorias a fazer	23
5.4	Conclusão	23
	Referências	25
A	Exemplo de apêndice	27

Capítulo 1

Introdução

1.1 O Jogo

O *Card Based Dungeon Crawling Game* é inspirado em vários conceitos já muito explorados no que diz respeito a jogos, tanto a nível físico como digital:

- **Jogo baseado em Cartas:** num jogo baseado em cartas, a maioria dos elementos da experiência podem, e muitas vezes são representados através de cartas. Jogos familiares como UNO! [1] ou Trading Card Games como Magic: The Gathering [2] são excelentes exemplos deste tipo de jogo.
- **Dungeon Crawler:** um dungeon crawler é um género de jogo baseado na exploração de uma área repleta de inimigos hostis ao jogador, tesouros, e objetivos que devem ser cumpridos. Diferente dos jogos baseados em cartas, é difícil encontrar exemplares físicos deste tipo de jogo, mas há vários videojogos, como The Binding of Isaac [3] por exemplo, que são bons representantes do género.
- **Jogos por Turnos:** num jogo por turnos, ao invés de todos os jogadores jogarem ao mesmo tempo, cada um tem um momento alocado para agir. Jogos como Xadrez [4], na vertente de jogos de tabuleiro, ou o combate em Baldur's Gate [5], no que diz respeito a Dungeon-Crawlers, são excelentes exemplos deste género.

No CBDCG, um jogador terá um conjunto de personagens à sua disposição, e terá de construir e percorrer o tabuleiro de jogo juntamente com os restantes jogadores.

Cada personagem tem um conjunto de atributos e habilidades diferentes, o que exige uma mentalidade e estilo de jogo diferente por parte do jogador. Os quatro atributos principais de um personagem são:

- **Pontos de Vida (HP):** como o nome indica, são os pontos de vida de um personagem. Quando este valor chega a 0 durante a exploração da área de jogo, ele é considerado incapaz de continuar e é removido. Quando este valor chega a 0 durante uma batalha, o personagem

é derrotado e perde um ponto permanentemente.

- **Ataque (DMG):** dano que o personagem causa num turno de uma batalha.
- **Defesa (DEF):** dano que o personagem mitiga num turno de uma batalha.
- **Velocidade (SPE):** determina a ordem de jogo, com os personagens mais rápido a agir primeiro, ou a ordem de ataque dos personagens numa batalha. Além disso, influencia na probabilidade de um personagem fugir de uma batalha, ou passar despercebido durante a exploração.

Os personagens também são capazes de utilizar itens para se fortalecerem, ou podem ativar efeitos especiais durante a exploração do tabuleiro que aumenta os seus atributos, ou diminui os atributos dos oponentes.

O objetivo do jogo é simples: derrotar os personagens de todos os outros jogadores, ou obter todos os anéis e voltar ao início da masmorra. Assim sendo, apenas um jogador pode vencer este jogo.

1.2 Como Jogar

Como a maior parte dos jogos de tabuleiro, cada jogador terá um conjunto de ações que pode realizar durante o seu turno:

- **Fase de Construção:** o jogador retira uma carta do baralho das peças do tabuleiro e coloca-a na sua mão. Além de não ser sempre obrigado a colocar uma peça no tabuleiro, durante o seu turno, o jogador pode colocar qualquer número de peças de uma só vez, desde que todas liguem ao resto do tabuleiro.
- **Fase de Substituição:** o jogador decide se quer continuar com o seu personagem, ou se prefere trocar para outro personagem na sua mão. O jogador também pode equipar ou desequipar qualquer número de Itens de qualquer um dos seus personagens.
- **Fase de Movimento:** o jogador pode mover o seu personagem. Antes de passar por uma peça com um efeito especial, este é automaticamente ativado. Personagens podem desafiar outros personagens adjacentes. Qualquer personagem adjacente a um dos combatentes também se pode juntar à batalha.

Durante uma batalha, um jogador pode executar uma de três ações com o seu personagem:

- **Atacar:** o personagem tenta atacar um oponente. O valor de DMG do atacante é reduzido primeiro ao valor de DEF do defensor, e depois ao de HP.
- **Aguardar:** esta ação permite que o personagem assuma uma postura defensiva, aumentando a sua DEF em 1 ponto, ou recuperando a sua armadura para o valor original. Esta ação não pode ser usada consecutivamente.

- **Fugir:** esta ação representa uma tentativa de abandonar o combate. A probabilidade de sucesso vai aumentar ou diminuir dependendo da diferença entre o valor de SPE do personagem e o valor de SPE do oponente com maior velocidade.

1.3 Organização do Documento

O restante relatório encontra-se organizado da seguinte forma:

Capítulo 2

Enquadramento

2.1 Trabalho relacionado

2.2 Sistemas semelhantes

Capítulo 3

Arquitetura

3.1 Tecnologias usadas

Kotlin Multiplatform

Kotlin Multiplatform [6], ou KMP, é uma framework desenvolvida pela JetBrains [7] que permite desenvolver código comum a várias plataformas utilizando apenas Kotlin.

Um projeto multiplatform é dividido em três módulos principais: *composeApp* (o Cliente), *server* (o Servidor) e *shared*.

O módulo *shared* possui código que será compartilhado entre cliente e servidor, essencial para evitar escrever código duplicado, e para permitir uma comunicação simplificada entre cliente e servidor, uma vez que ambos têm acesso aos recursos deste módulo.

O módulo *composeApp* possui uma implementação do cliente para cada plataforma (JVM, Android, iOS, JS, ...), e código que é compartilhado entre todos os clientes, uma vez mais, para diminuir a quantidade de código repetido.

O módulo *server* possui a arquitetura do servidor **não sei o que dizer aqui sem falar de detalhes de implementação...**

O facto do cliente e servidor partilharem os mesmos elementos de domínio permite testar os três componentes individualmente, sem a utilização de Mocks complexos, e os testes podem ser facilmente escritos de modo a serem unitários e determinísticos.

Ktor

Uma vez que estamos a usar Kotlin Multiplatform, faz sentido utilizar a biblioteca Ktor [8] para criar aplicações Cliente e Servidor, uma vez que também foi um produto produzido pela JetBrains.

Esta biblioteca permite a adição de funcionalidades, como por exemplo, routing, serialização de dados, ou tratamento de exceções, de forma modular através da instalação de plugins, ou seja, permite criar um cliente ou servidor extremamente configurável. Graças a isso, cliente e servidor são capazes de comunicar tanto via pedidos e respostas HTTP, como através de sinais WebSocket.

O ktor utiliza coroutines do Kotlin como mecanismo de concorrência, o que significa que permite a execução de operações de forma assíncrona. Além disso, também tira proveito de outros elementos do ecossistema Kotlin, como a biblioteca de serialização kotlinX e o próprio KMP.

Exposed

Para realizar implementação e comunicação, com a base de dados, escolhemos a biblioteca [?]. Ao contrário de algumas outras tecnologias, esta não foi escolhida de todo por termos familiaridade com a mesma, mas sim completamente pela utilidade que oferece, tendo sido a falta de familiaridade o maior desafio a utilizar a mesma. As várias vantagens desta biblioteca aplicam-se a qualquer projeto Kotlin que interaja com bases de dados SQL. A maior vantagem e a principal razão da existência da biblioteca, é que nos permite desenvolver código de acesso à base de dados em Kotlin, ou seja, o código será compilado ao mesmo tempo que o resto do projeto, evitando potenciais erros de runtime, ou seja, a Domain Specific Language do Exposed torna as queries Type-Safe, impedindo que todos os dados estão corretos antes do programa correr.

As outras vantagens estão no mapeamento objeto-relacional ([?]ORM) utilizado para interagir com a base de dados, ou seja, para além de a linguagem utilizada ser type-safe e compilada em compile time, os objetos da base de dados encontram-se mapeados na forma de objetos do domínio, simplificando o desenvolvimento. A framework centrada á volta de objetos de acesso a dados ([?]DAO), contribui também para o fácil desenvolvimento, simplificando o acesso aos dados, definindo os objetos para acesso aos mesmos. Consequentemente mas ainda não mencionado de forma direta, todo o acesso a dados e defenição da base de dados é realizado em linguagem kotlin, um beneficio também extremamente grande para um projeto que já utiliza kotlin em todas as partes do mesmo.

As alternativas a esta biblioteca que inicialmente consideramos, por sermos mais familiares com as mesmas eram jdbi ou jdbc. Estas bibliotecas são relativamente semelhantes sendo jdbi aquela que provavelmente teríamos escolhido em vez de exposed devido à sua melhor aplicação a projetos maiores e maior abstração a escrever queries. No entanto apesar das diferenças estas bibliotecas têm ambas o mesmo grande problema. Obrigam a utilização de queries em psql para comunicar com a base de dados. Como foi mencionado, esta é a grande

vantagem do exposed, e uma que não é encontrada nas suas alternativas de bibliotecas que gerem interações com bases de dados. A vantagem de código 100% kotlin, focado em objetos e compilado ao mesmo tempo que todo o projeto, é demasiado grande para considerarmos qualquer outra opção.

Dado tudo isto, a biblioteca exposed foi a escolha lógica para o nosso projeto, apesar da falta de familiaridade, é uma tecnologia recente, e melhor do que qualquer alternativa encontrada.

Jetpack Compose

Dado que a framework utilizada foi Kotlin Multiplatform, também temos a capacidade de utilizar Compose Multiplatform para a partilha da interface gráfica entre vários clientes distintos. Assim, basta realizar apenas uma iteração da interface em Kotlin, que a framework permite a execução nativa em diferentes plataformas, criando uma experiência semelhante para qualquer tipo de cliente com o mesmo pedaço de código.

O Jetpack Compose [9] tem a vantagem de ser um modelo reativo e modular, o que significa que uma alteração no estado da aplicação pode provocar recomposições no ecrã de determinados componentes, e noutros não. A modularidade também é importante porque permite a reutilização de componentes do ecrã.

Reflect?

No nosso projeto utilizamos a biblioteca reflect, para construir uma estrutura polimorfa, que reflete sempre todos os membros dessa estrutura que se encontram implementados.

Esta biblioteca foi utilizada por garantir que adicionar membros a esta estrutura, apenas é necessário construir dito membro, que é automaticamente detetado e mapeado como todos os outros, expondo a função ao resto da aplicação. Utilizar reflexão foi extremamente útil, pois permite implementar a estrutura polimorfa desejada, com o mínimo de esforço necessário para adicionar membros á mesma. Não utilizando, reflexão seria necessário adicionar muito mais lógica por cada membro adicionado.

3.2 Aplicações cliente

3.3 Aplicação servidor

O servidor do projeto, ???? kinda ns oq dizer n ha mt ne?

3.4 Base de dados

Para armazenar, gerir, e organizar informação de forma persistente, recorreremos à utilização de uma Base de Dados PostgreSQL [?]. Utilizando a biblioteca Exposed anteriormente mencionada, somos capazes de consultar, atualizar, ou remover informação da base de dados. Dado que

3.5 Requisitos Funcionais

Gestão de Utilizadores

- Criação de conta
- Autenticação (Login/Logout)

Gestão de Mesas de Jogo

- Criação de uma mesa
- Listagem de todas as mesas
- Entrada e Saída de uma mesa
- Alteração do papel a desempenhar no jogo (Player/Spectator)

Lógica de Jogo

- Colocação de Peças e Personagens na área de jogo
- Equipar Itens a um Personagem
- Rotação de peças na mão do jogador
- Movimentação de Personagens na área de jogo
- Capacidade de ativar efeitos de uma Peça
- Trocar Personagens e desequipar Itens
- Sistema de Combate
- Implementação das habilidades passivas dos Personagens
- Verificação das condições de vitória

Capítulo 4

Implementação

4.1 Organização do projeto

4.1.1 Compose App

Este módulo contém a lógica do cliente e os elementos de composição da User Interface. Subdivide-se em:

- commonMain: componentes comuns a qualquer cliente da aplicação;
- jvmMain: cliente JVM (Desktop);
- wasmJsMain: cliente wasmJS (Browser);

O módulo compartilhado subdivide-se ainda em:

- navigation: rotas de ecrã e componente de navegação entre ecrãs do cliente;
- viewmodel: componente intermédio entre o estado recebido do servidor e a UI do cliente;
- views: componentes visuais do cliente;

4.1.2 Server

Este módulo contém a arquitetura do servidor, e trata de expor os meios de comunicação entre o mesmo e os clientes, implementando a forma como a lógica da aplicação pode ser utilizada. Contém também a camada de acesso a dados: Subdivide-se em:

- Config: Ficheiros de configuração;
- Repository: Camada de acesso a dados (DAL);
- Service: Comunicação servidor-Domínio;

- WebApi: Comunicação cliente-servidor;
- Application: Ficheiro que controla e inicia o servidor;

4.1.3 Shared

No modulo shared, encontra se todo o código que é partilhado entre cliente e servidor. Este modulo é uma das grandes vantagens que vem com a utilização de KMP, pois através deste, é eliminada a necessidade de ter múltiplas definições das entidades do domínio em ambos os lados do código, simplificando muito a estrutura do projeto.

Este modulo contém a lógica da aplicação, definições de objetos utilizados na mesma, e objetos de comunicação com o cliente. Por ser uma aplicação que utiliza KMP, todo este código é partilhado com toda a aplicação, daí o nome shared, implementa toda a lógica partilhada por toda aplicação. É composto por:

- Domain: Lógica da aplicação e definições de objetos;
- DTO: Objetos de comunicação com o cliente;
- Error: Definição dos erros lançados durante a aplicação;

4.2 FrontEnd/Cliente

A nossa aplicação tem dois tipos de cliente Ktor: Desktop (JVM) e Browser (wasmJS). Ambos usam uma engine assíncrona e instalam plugins para as comunicações HTTP e WebSocket, e para o armazenamento de recursos (imagens), sendo que o cliente Desktop tem a capacidade de guardar estes recursos permanentemente através do FileStorage, criando uma pasta local para armazenamento, e o cliente Browser está dependente do próprio browser.

4.2.1 Navegação

A navegação entre ecrãs foi implementada recorrendo à biblioteca Navigation Compose do Jetpack Compose. Há três componentes principais no que diz respeito à navegação entre ecrãs:

- NavHost: é o contentor que associa o controlador ao conjunto de destinos disponíveis ao cliente.
- NavController: o controlador é o componente responsável por controlar o fluxo da navegação, isto é, a navegação entre estados e a manipulação da pilha de navegação.
- Rotas: conjunto de destinos que o controlador tem acesso, cada um associado a um elemento visual.

Estes componentes, no entanto, não são suficientes para mudar o estado do cliente. Como foi referido anteriormente, o Jetpack Compose é reativo, ou seja, é capaz de detetar e reagir às variações de um `StateFlow`. Um `StateFlow` mantém o valor mais recente de um conjunto de valores emitidos ao longo do tempo de forma assíncrona, ou seja, à medida que são realizadas alterações no servidor, estas alterações são enviadas ao cliente e alteram o `StateFlow`. O cliente está preparado para coletar este valor recém-chegado e verificar em que rota é que o controlador deve estar.

O cliente tem cinco possíveis rotas, ou seja, cinco possíveis ecrãs de destino:

- Menu: o ecrã inicial da aplicação, mostrado quando um cliente não está autenticado;
- Tutorial: o ecrã destinado à execução de determinado tutorial;
- Lobby: o ecrã onde o cliente pode criar ou juntar-se a mesas de jogo;
- Table: o ecrã onde o cliente define o seu papel no jogo, e espera que o mesmo comece;
- Game: o ecrã apresentado durante a execução de um jogo;

Deste modo, há uma divisão de responsabilidades clara, dado que a navegação não depende totalmente da interação de um utilizador com a UI, mas sim de uma alteração no estado da aplicação, permitindo atualizar o ecrã em tempo real. Dito isto, ainda há navegação "manual", isto é, dependente de um clique do utilizador, como por exemplo, voltar ao ecrã anterior, ou fechar uma janela de erro, uma vez que estes podem não exigir que seja feito um pedido ao servidor por ser uma operação exclusiva do cliente.

4.2.2 ClientAPI

O `ClientAPI` é o módulo que abstrai o resto do cliente de qualquer interação com o servidor. É este componente que trata de construir e realizar os pedidos HTTP ao servidor, e transforma a resposta numa alteração de estado. De forma semelhante, o `ClientAPI` também faz a gestão da comunicação WebSocket, e está preparado para receber e decodificar sinais específicos vindos do servidor.

O `ClientAPI` mantém três `StateFlows`:

- `tables`: variável que armazena todas as mesas disponíveis para entrar.
- `currentTable`: variável que armazena a informação da mesa frequentada pelo cliente.
- `game`: variável que armazena a informação do jogo em que o cliente está a participar.

Estes três `StateFlows` sofrem alterações quando o cliente recebe um sinal WebSocket vindo do servidor. A comunicação HTTP, não sendo assíncrona, não influencia os valores dos `StateFlows`:

- LobbyTables: quando uma mesa é criada, eliminada, ou alterada, é necessário atualizar 'tables'.
- TableInfo: quando a mesa em que o jogador está é alterada, é preciso atualizar 'currentTable'.
- TableDeleted: quando a mesa em que o jogador está é eliminada, é preciso atualizar 'currentTable'.
- GameInfo: quando o jogo é atualizado, é preciso atualizar 'game'.

4.2.3 ViewModel

4.2.4 Ecrãs

4.3 BackEnd/Servidor

4.3.1 Base de dados

A base de dados do projeto foi implementada com o uso da biblioteca exposed, para definir as tabelas que compõem a BD e para comunicar com a mesma.

Esta segue na sua maioria um modelo relacional, no entanto existem alguns objetos armazenados como json, ou seja são opacos à base de dados. Isto, encontra-se apenas presente para os objetos Game, que têm uma complexidade bastante alta, e que necessitam de uma grande quantidade de informação persistida por cada um. Foi por este motivo, que implementamos a representação deste objeto na base de dados desta maneira: para facilitar o desenvolvimento. A implementação reduz o número de tabelas necessário, pois os objetos guardados em json foram apenas aqueles que tinham uma difícil representação e que, têm relação 1-1 com a tabela jogo, isto diminui a quantidade de queries e lógica necessária para obter e guardar um jogo da base de dados.

No entanto é necessário admitir que esta implementação é problemática e que a longo prazo os problemas são maiores que os benefícios, pois impossibilita de no futuro realizar pesquisa baseada em qualquer um dos objetos guardados como Json sem deserializar e serializar o objeto primeiro, ou alterar a definição da base de dados.

Por este motivo apesar da simplicidade oferecida por esta implementação e do tempo ganho para focar em outras partes do projeto, gastando menos tempo na base de dados em si tanto na implementação inicial como a implementar subsequentes mudanças aos objetos representados dessa maneira, é um facto que a implementação deveria ser melhorada.

A biblioteca exposed através da sua dsl trata de toda a interação com a base de dados sem necessitarmos de escrever quaisquer queries sql, trata também da serialização e deserialização dos objetos json armazenados. Foram realizados testes a base de dados, que foram princi-

palmente utilizados para desenvolver a implementação inicial da mesma, no entanto alguns destes encontram-se desatualizados, pois não voltou a surgir a necessidade de os utilizar. É verdade que devíamos ter os testes atualizados, no entanto estes foram úteis na mesma. Futuras versões da base de dados deveriam ter ambos esses pontos corrigidos, a base de dados deverá ser 100% relacional, e os testes à mesma deverão estar atualizados.

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do utilizador
name	VARCHAR(255)	NOT NULL	Nome de exibição do utilizador
email	VARCHAR(255)	UNIQUE, NOT NULL, Índice	Endereço de email do utilizador
password	VARCHAR(255)	NOT NULL	Palavra-passe encriptada
creation_date	LONG	NOT NULL	Data de criação da conta

Tabela 4.1: Tabela Users

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único
token	VARCHAR(255)	UNIQUE, Índice	Token de autenticação
user_id	INT	Chave Estrangeira (Users.id), UNIQUE, Índice	Referência ao utilizador
game_id	INT	Nullable	Referência ao jogo
token_refresh_time	LONG	NOT NULL	Data da última atualização
token_expiration	LONG	NOT NULL	Data de expiração

Tabela 4.2: Tabela AuthUsers

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único da mesa
name	VARCHAR(255)	UNIQUE, NOT NULL, Índice	Nome da mesa/sala de jogo
owner	INT	Chave Estrangeira (Users.id), NOT NULL	Utilizador que criou a mesa
capacity	INT	NOT NULL	Número máximo de jogadores

Tabela 4.3: Tabela Tables

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do participante
user_id	INT	Chave Estrangeira (Users.id), NOT NULL	Referência ao utilizador
lobby_id	INT	Chave Estrangeira (Tables.id), NOT NULL	Referência à mesa de jogo
role	VARCHAR(10)	NOT NULL	Papel do jogador (PLAYER/SPECTATOR)

Tabela 4.4: Tabela Participants

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do jogo
turn	JSON	NOT NULL	Estado do turno atual (TurnDTO)
item_deck	JSON	NOT NULL	Array de itens no baralho
tile_deck	JSON	NOT NULL	Array de peças no baralho
battle	JSON	Nullable	Estado da batalha atual (BattleDTO)

Tabela 4.5: Tabela Games

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do jogo
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
user_id	INT	Chave Estrangeira (Users.id), CASCADE	Referência ao utilizador
player_hands	JSON	NOT NULL	Array de cartas na mão
current_character	VARCHAR(255)	Nullable	Nome da personagem

Tabela 4.6: Tabela GamePlayers

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do espectador
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
user_id	INT	Chave Estrangeira (Users.id), CASCADE	Referência ao utilizador

Tabela 4.7: Tabela GameSpectators

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único da peça
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
x	INT	NOT NULL	Coordenada X no tabuleiro
y	INT	NOT NULL	Coordenada Y no tabuleiro
board_tiles	JSON	NOT NULL	Dados da peça (TileDTO)
character_name	VARCHAR(255)	Nullable	Nome da personagem no jogo

Tabela 4.8: Tabela BoardTiles

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do item
board_tile_id	INT	Chave Estrangeira (BoardTiles.id), CASCADE	Referência à peça do tabuleiro
name	VARCHAR(255)	NOT NULL	Nome do item

Tabela 4.9: Tabela BoardCharacterItems

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do modificador
character_name	INT	Chave Estrangeira (BoardTiles.id), CASCADE	Referência à peça do tabuleiro
stats	VARCHAR(50)	NOT NULL	Estatísticas afetadas
duration	INT	NOT NULL	Duração do modificador

Tabela 4.10: Tabela BoardCharacterModifiers

4.3.2 Dominio

4.3.3 Expansibilidade

Um grande foco na nossa implementação foi a expansibilidade do código, foi claro desde o início que implementar tudo o que tínhamos planejado seria difícil, então decidimos colocar o foco em garantir que melhorias e adições futuras seriam o mais simples possível, para esse efeito foram tomadas várias decisões de design que apesar de terem complicado a implementação tinham o propósito de tornar o código facilmente expansível. Polimorfia em conjunção com reflexão foram a principal maneira de cumprir este objetivo, para além disso, com estas duas ferramentas, e desenhando o código de maneira a que quaisquer adições de funções que causam modificações simples ao jogo, podem ser adicionadas sem a necessidade de criar novas rotas. Para além disso como já foi mencionado, o desenho geral do domínio está também construído para este propósito, mantendo as definições dos objetos de forma o mais geral possível tentando reduzir o número de objetos principais, e reduzindo informação redundante em cada um que complicaria expandir os mesmos.

Polimorfia

Polimorfia foi extremamente útil para o propósito de fazer o código expansível, utilizamos técnicas polimórficas para:

A abstração Entity: Utilizada para representar diversos objetos do jogo sobre o mesmo tipo. É a base dos principais usos de polimorfia no projeto, Foi essencial para permitir que as Interfaces polimórficas implementadas possam afetar o máximo de objetos possível, expandindo os usos possíveis das mesmas. Esta abstração engloba os objetos:

- Player;
- BoardTile;
- Board;
- Character: Inclui todos os tipos de personagem atual (PlayableCharacter) e futuros como personagens inimigos controlados automaticamente;
- Card: Inclui todos os tipos de carta;
- BattleAction;
- Battle;
- Game;

Definir funções que causam mudanças simples ao jogo de forma geral através da interface UpdaterE e da abstração Entity, esta interface é uma interface funcional, permitindo que novas implementações sejam ainda o mais simples possível;

Na definição das cartas de jogo sendo possível adicionar novos tipos de carta;

Nas passivas dos personagens, sendo possível implementar passivas que afetam qualquer objeto que pode ser representado pela abstração Entity tal como updaterE esta interface é uma interface funcional, pela mesma razão que updaterE é uma interface funcional, simplifica a implementação e facilita a leitura. É graças à existência de Entity, e ao desenho do domínio que possibilita que futuros personagens implementados tenham passivas que vão desde adicionar simples Modifiers que afetam as suas Stats, até afetar itens de outros jogadores, ou mesmo gerar TileEffects. Futuras implementações poderiam até ter habilidades que são ativadas pelos próprios jogadores, sendo apenas necessário criar uma função para ativar as mesmas, algo que seria relativamente simples graças à existência de UpdaterE, comparando à dificuldade de implementar a mesma funcionalidade sem a existência de UpdaterE e Entity. Infelizmente não tivemos tempo para implementar exemplos de todas estas passivas mais complexas, apesar do foco em simplificar a implementação das mesmas. No entanto foi ter noção desta dificuldade que nos levou a fazer implementar futuras passivas com diversidade extremamente alta, o mais simples possível, pois um qualquer programador, seria capaz de desenvolver as mesmas sem necessariamente ser familiar com todo o projeto.

Reflexão

Reflexão foi utilizada para que todas as funções classificadas como UpdaterE possam ser detetadas em compile time, e associadas ao seu nome, para que seja possível chamar ditas funções apenas com o seu nome reduzindo o número de rotas necessárias. Graças a reflexão e à abstração Entity com apenas uma única rota é possível definir quaisquer funções de jogo desde que tenham dois ou menos tipos que possam ser representados por Entity e que devolvam um jogo. Neste momento existem 12 funções que de outra forma necessitariam de rotas para cada uma ou de associar cada uma ao seu nome manualmente.

4.3.4 Autenticação

A nossa autenticação segue um esquema desenvolvido por nós para se aplicar da melhor forma possível às necessidades do nosso projeto.

No contexto do nosso projeto, a autenticação e segurança serve principalmente o papel de garantir que nenhum utilizador é capaz de interagir com um jogo de forma maliciosa. Tirando isso, a autenticação é apenas utilizada para identificar quais utilizadores é que estão logged-

in, impedindo tentativas de log in enquanto uma sessão atual permanece ativa. Removemos também de forma automática e periódica os utilizadores cuja sessão terminou, no entanto, não realizamos isso se o utilizador se encontrar dentro de um jogo que ainda não terminou.

Explicando a autenticação em si, fora do jogo apenas é verificado se o utilizador está autenticado ou não, com o parâmetro `auth` do mesmo. Este encontra-se em `null` sempre que o utilizador perder o log in por qualquer razão.

Quando um jogador entra no jogo, a autenticação passa a ser validada com uma combinação do token de autenticação e de um id de jogo e tempo de fim de sessão. A id do jogo tem de ser equivalente aquela do jogo em que o jogador entrou. É também utilizado um `Instant` para identificar quando o token deve ser substituído por um novo garantindo que token válido é substituído de forma periódica. Este `Instant` intitulado de `refreshTime` também foi considerado ser utilizado, para identificar inatividade. Alterando-o com todas as ações de jogo, se tiver um valor muito antigo, isso implica um cliente inativo. É possível então, através desse valor calcular então exatamente quanto tempo passou desde a última ação de jogo que um qualquer cliente realizou e utilizar esse tempo para definir um máximo possível aceitável para este se encontrar inativo em jogo, e fora de jogo, sendo estes estados facilmente identificados com o parâmetro `gameId` que é `null` quando o jogador está fora de um jogo. Isto poderia ser realizado tanto com o dito `refresh` deste valor, mas mais corretamente para impedir o token de ser substituído a cada ação, seria melhor realizado utilizando um terceiro instante para o propósito de identificar inatividade.

Apesar de ser menos ortodoxa do que uma implementação de autenticação padrão, utilizando por exemplo bibliotecas como o `JWT` ou mesmo aplicando uma mais autenticação simples com apenas tokens e fim de sessão, que já implementamos em vários projetos durante o curso, para o nosso projeto acreditamos que funciona tão bem ou melhor do que qualquer outro, pois tem os benefícios de garantir a integridade do jogo, enquanto se mantém simples na sua implementação, e permite obter na mesma bastante informação sobre a sessão de um qualquer cliente dependendo das necessidades. Escolhemos implementar nós um sistema de autenticação exatamente para que esta tenha benefícios explicitamente para o nosso projeto, sendo os benefícios que ganhamos com esta, na nossa opinião, maiores do que quaisquer benefícios que um sistema mais “padrão” pudesse oferecer. Isto, pois, necessidade de autenticação vai sempre ser diferente de caso em caso, e apesar de no geral estas necessidades serem semelhantes para a grande maioria dos casos, não são semelhantes para o nosso. Não existe grande necessidade de garantir integridade fora de um jogo, e desde que o nosso sistema dê fácil acesso à informação da sessão dos clientes, especialmente sobre a sua atividade para remover aqueles que se encontram inativos, é um bom sistema. Apesar de isto poder ser considerado um problema de segurança, não é um que consideramos grave, nem que valia a pena ser resolvido no contexto

do nosso projeto. Como foi mencionado pensamos em alternativas deste sistema, e ter um sistema pouco complicado, ajuda a surgindo a necessidade, como por exemplo de identificar utilizadores inativos, alterar o sistema para satisfazer dita necessidade.

4.3.5 Comunicação cliente-servidor

A comunicação entre cliente e servidor, é realizada através das rotas expostas pelo servidor que definem a API da aplicação. Os clientes utilizam a mesma para comunicar com o servidor e realizar as operações desejadas. São utilizados data transfer objects (DTOS), representações serializáveis de objetos do domínio necessários na comunicação ou conjunto de informação necessária para cada pedido. Estes ajudam a generalizar a informação necessária, e minimizam os dados enviados, pois cada objeto apenas contém o extritamente necessário para representar um qualquer objeto do domínio, sendo o resto da informação calculada. Do lado do cliente foi implementada a clientAPI, o modulo responsável por determinar quais rotas são definidas para quais ações. Desta forma, para mudar qualquer rota, é apenas necessário alterar qual é chamada em clientAPI, e todas as funções que necessitam dessa ação irão utilizar dita rota. Isto simplifica alterações quer na API da aplicação, quer na definição de cada ação, pois a logica de acesso ao servidor se encontra centralizada.

As rotas que definem a API da nossa aplicação são:

- **Game API (/game)**
 - POST /game/create
 - POST /game/place
 - POST /game/rotate
 - POST /game/end-turn
 - POST /game/leave
 - POST /game/unequip
 - POST /game/applyGameUpdater
- **Table API (/tables)**
 - GET /tables
 - POST /tables/create
 - POST /tables/join
 - POST /tables/leave
 - POST /tables/change-role
- **User/Auth API (/auth/users)**
 - POST /auth/users/create
 - POST /auth/users/login
 - POST /auth/users/logout
- **WebSocket API (/ws)**
 - websocket /ws

4.3.6 Testes

A implementação de testes unitários ao projeto, foram um dos pontos do projeto que podia ter sido muito melhorado. Foram desenvolvidos testes, e certas partes do projeto têm boa cobertura, no entanto outras encontram-se ou sem testes, com testes desatualizados ou com testes que têm baixa cobertura. Nos locais que contêm ou chegaram a conter boa cobertura os testes foram úteis, no entanto manter os testes atualizados foi algo que não recebeu a atenção necessária. Apesar de não termos visto a necessidade de colocar esforço na implementação de testes, é um facto que este ponto deveria ser melhorado. Os pontos do projeto em que testes foram mais utilizados e mais úteis foram:

- Para testar o funcionamento das funções construídas de forma polimorfa;
- No teste do repositório base de dados;
- Para algumas das funcionalidades do jogo;

Apesar de nestes pontos os testes foram úteis alguns, acabaram por se encontrar eventualmente desatualizados, pois depois da sua utilização inicial, não foi vista a necessidade de os atualizar. Dado isto em futuras atualizações do projeto, seria ideal corrigir este problema, aumentando a cobertura dos testes para cobrirem todos os pontos da aplicação necessários.

Capítulo 5

Conclusões

- 5.1 Dificuldades no projeto
- 5.2 Objetivos não implementados
- 5.3 Melhorias a fazer
- 5.4 Conclusão

Referências

- [1] Mattel. Uno! official website, 2026. Accessed July 9, 2026.
- [2] Wizards of the Coast. Magic: The gathering official website, 2026. Accessed July 9, 2026.
- [3] Edmund McMillan. The binding of isaac official website, 2026. Accessed July 9, 2026.
- [4] International Chess Federation (FIDE). Fide official website, 2026. Accessed July 9, 2026.
- [5] Larian Studios. Baldur's gate 3 official website, 2026. Accessed July 9, 2026.
- [6] JetBrains. Kotlin multiplatform, 2026. Accessed July 9, 2026.
- [7] JetBrains. JetBrains official website, 2026. Accessed July 9, 2026.
- [8] JetBrains. Ktor official website, 2026. Accessed July 9, 2026.
- [9] JetBrains Google. Jetpack compose, 2026. Accessed July 9, 2026.

Apêndice A

Exemplo de apêndice

Este é o primeiro parágrafo do apêndice.

Etiam ac leo a risus tristisque nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristisque senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

Nulla mattis luctus nulla. Duis commodo velit at leo. Aliquam vulputate magna et leo. Nam vestibulum ullamcorper leo. Vestibulum condimentum rutrum mauris. Donec id mauris. Morbi molestie justo et pede. Vivamus eget turpis sed nisl cursus tempor. Curabitur mollis sapien condimentum nunc. In wisi nisl, malesuada at, dignissim sit amet, lobortis in, odio. Aenean consequat arcu a ante. Pellentesque porta elit sit amet orci. Etiam at turpis nec elit ultricies imperdiet. Nulla facilisi. In hac habitasse platea dictumst. Suspendisse viverra aliquam risus. Nullam pede justo, molestie nonummy, scelerisque eu, facilisis vel, arcu.